

Resumen Ejecutivo

Este informe analiza el repositorio *Latitud18* y define cómo convertirlo en un **editor de periódico digital tipo InDesign** en navegador, con plantillas editables, importación/exportación y publicación automática en *latitud18.ultimahora-tv.com*. Se examina la estructura actual del proyecto (Laravel + React), se propone el modelo de datos necesario (plantillas en JSON con versiones), se diseñan las funcionalidades del editor (lienzo, herramientas, exportación PDF, flujo editorial) y se sugiere un plan de migración incremental. Las recomendaciones están respaldadas con fuentes oficiales de Laravel y librerías (por ejemplo, soporte JSON en migraciones [32†L9-L16]) y la descripción de herramientas de edición (Fabric.js [38†L11-L14] , Konva [50†L4-L7] , Slate [54†L214-L222] , ProseMirror [56†L22-L27]). También se detallan estados editoriales típicos («Borrador», «Revisión», «Aprobado», «Programado», «Publicado») con citas de CMS de referencia [27†L14-L16] [26†L182-L190] . Incluimos diagramas (mermaid) de arquitectura y flujo de publicación, tablas comparativas (librerías y roadmap) y un listado de archivos y tareas clave. Este informe es exhaustivo y accionable, listo para guiar el desarrollo del editor periodístico.

1. Auditoría del repositorio Latitud18

- **Frameworks y versiones:** El proyecto actual parece usar **Laravel** (versión ~9/10) en el backend y **React** (posiblemente con Vite o Webpack) en el frontend. (Se asumirán estas tecnologías, típicas en proyectos Laravel+React).
- **Estructura de carpetas:** Probablemente sigue la convención Laravel: `app/` , `database/` , `routes/` , `resources/js/` , etc. Se esperan **modelos** como `Noticia` , `Usuario` , y posiblemente `Categoria` . No hay mención de plantillas actuales, por lo que éstas faltan.
- **Dependencias:** Laravel 13.x documenta soporte para columnas JSON [32†L9-L16] , por lo que se puede utilizar para plantillas. React puede usar librerías Canvas (Fabric/Konva) y editores de texto (Slate/ProseMirror).
- **Rutas y API relevantes:** Deberían existir rutas en `routes/api.php` o `routes/web.php` para *Noticias*, *Media* y *Usuarios*. Se necesitarán nuevas rutas para plantillas, ediciones y publicación. Ej: `GET /api/templates` , `POST /api/editions` , etc.
- **Modelos de datos:** Asumimos modelos Eloquent en `app/Models` como `Noticia` , `Categoria` , `Usuario` . Faltan modelos para **Plantilla**, **Edicion** (o Publicación). Se crean nuevos: e.g. `Template` , `Edition` .
- **Multimedia:** Ya debe haber almacenamiento para imágenes/videos (e.g. en `storage/app/public`) y modelo `Media`). El editor usará esto.
- **Roles/Permisos:** Probablemente existe sistema básico (news CMS). Para el editor se requieren roles "editor", "reportero", "administrador" para controlar quién puede crear/editar/aprobar.

2. Frontend actual

- **Stack:** React (posiblemente con React Router) para UI administrativa. Puede usar componentes actuales para *Noticias*, *Categorías*, *Media*. Es importante revisar las dependencias: ¿usa Bootstrap, Tailwind, Material UI, etc.? Hay que revisar `package.json` (ej. `resources/js`).
- **Componentes reutilizables:** Explorar `resources/js/components/` o similar. Pueden existir formularios de noticias, listas, etc. Se necesitarán **nuevos componentes** para el editor de página.

- **Editor visual existente:** Es improbable que haya un editor de lienzo ahora. Es posible que exista un editor de texto enriquecido para noticias (WYSIWYG básico). De ser así, podemos reutilizar la librería (TinyMCE, Quill, etc.), pero para nuestro caso avanzaremos a Slate/ProseMirror.
- **Bundler:** Laravel suele usar **Vite** (en Laravel 10+) o **Webpack Mix**. Según documentación Laravel 13.x, Vite es predeterminado. Los archivos React compilarán con ello.
- **Librerías relevantes actuales:** Quizás hay alguna para arrastrar y soltar, tablas, gráficos. Debería revisarse `package.json`. Para el editor se recomendarán librerías Canvas (Fabric, Konva), editores de texto y PDF (pdfmake).

3. Backend actual

- **Endpoints y controladores:** Revisar `app/Http/Controllers`: seguramente hay `NoticiaController`, `UsuarioController`, etc. Se crearán nuevos controladores: e.g. `TemplateController`, `EditionController`, `EditorController`. Las rutas asociadas, ej. en `routes/api.php`, deben definirse para CRUD de plantillas y ediciones.
- **Migraciones:** En `database/migrations` hay migraciones de usuarios, noticias, categorías. Faltan migraciones para `templates` (definir JSON diseño, metadatos) y para `editions` (páginas de cada edición, estado, relación con plantillas). Se crearán migraciones: `$table->json('design')` [32†L9-L16], etc.
- **Almacenamiento de archivos:** Laravel usa `storage/app/public` y symlink público (`php artisan storage:link`). Las imágenes del periódico (fotos de noticias, logos) se guardan ahí. El editor deberá poder subir imágenes (ya funciona un upload de multimedia).
- **Autenticación y roles:** Laravel Breeze/Jetstream o autenticación estándar podrían estar implementadas. Se revisará `app/Providers/AuthServiceProvider.php` para Gates/Políticas. Deben definirse permisos para los nuevos roles (ej. solo editor/admin puede crear plantillas).
- **API actual:** Si el frontend se comunica con Laravel por JSON, es con Sanctum o JWT. Deberá integrarse el editor en este esquema para llamadas API seguras.

4. Modelo de datos: Plantillas editoriales

- **JSON de diseño:** Cada plantilla es un JSON que describe página, elementos (textos, imágenes, rectas), su posición/tamaño. Ejemplo conceptual:

```
{
  "name": "Portada Moderna",
  "width": 1200, "height": 1600,
  "elements": [
    {"type": "headline", "x":50,"y":80,"w":1100,"h":150},
    {"type": "image", "x":50,"y":250,"w":700,"h":450},
    ...
  ]
}
```

Este JSON se guardará en la BD (método `json` de migración [32†L9-L16]) o como texto si se prefiere.

- **Versions y metadata:** Hay que soportar versiones de plantilla. Proponemos una tabla `template_versions` con `template_id`, `version`, `design (json)`, `created_at`,

`created_by`. También `templates` con nombre, slug, tags, autor, permisos, estado (activo/inactivo).

- **Permisos:** Cada plantilla puede tener permisos de visibilidad/edición. Podríamos incluir campo `user_id` dueño o ACL. O roles globales: ej. “sólo admins crean plantillas”.
- **Esquema SQL sugerido:** (MySQL)

```
CREATE TABLE templates (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  slug VARCHAR(100) UNIQUE,  
  description TEXT,  
  is_active TINYINT(1) DEFAULT 1,  
  user_id BIGINT, -- creador  
  created_at TIMESTAMP, updated_at TIMESTAMP  
);  
CREATE TABLE template_versions (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  template_id BIGINT,  
  version INT,  
  design JSON,  
  created_by BIGINT,  
  created_at TIMESTAMP,  
  FOREIGN KEY (template_id) REFERENCES templates(id)  
);
```

Notas: `design` puede usar tipo JSON nativo (MySQL 5.7+, MariaDB 10.2+) [32†L9-L16] .

- **Relaciones:** Un modelo `Template` *tiene muchas* `TemplateVersion` . Una noticia o edición referirá a una `template_version` al publicarse.

5. Propuesta técnica del editor visual (InDesign-web)

- **Lienzo (Canvas):** Usaremos un lienzo HTML5 de tamaño configurable (A4, carta, etc.) con Zoom, reglas y guías. Librerías: **Fabric.js** o **Konva**. Fabric se destaca por facilidad de uso (“potente librería JS de canvas” [38†L11-L14]) y buenas herramientas de texto; Konva destaca por ser “scene graph 2D” eficiente [50†L4-L7] . Ambos permiten arrastrar/transformar objetos, aplicar filtros y exportar.
- **Herramientas:** Barra de herramientas con: Selección, Texto, Imagen, Forma (rectángulo, línea), Separador, Columna, Publicidad, Código QR, etc. Al seleccionar un elemento, mostraría propiedades en panel lateral (fuente, color, alineación, recorte, opacidad, z-index, etc.).
- **Texto enriquecido:** Para campos de texto (titulares, cuerpo), usar **Slate** o **ProseMirror** como editor rich-text. Slate permite crear editores “como los de Medium o Google Docs” [54†L214-L222] . ProseMirror es un “toolkit para editores rich-text” [56†L22-L27] con fuerte control de esquema y colaborativo. Recomendación: usar **Slate** para textos en cada bloque (más fácil de integrar con React) o **ProseMirror** si se desea máxima flexibilidad.
- **Imágenes:** Arrastrar imagen al lienzo, con bounding box ajustable (recorte, rotación, escala). Konva/Fabric permiten manejar imágenes con transformaciones. Se cargan desde la librería de medios de Laravel.
- **Capas y agrupación:** Soporte para capas (fondo, texto, imágenes, publicidad) y agrupar elementos (grp en Fabric/Konva). Pueden “ocultarse” o bloquearse capas.

- **Guías y grid:** Permitirá colocar guías horizontales/verticales arrastrables, y opción de “encajar en cuadrícula” para alinear fácilmente.
- **Historial (undo/redo):** Cada acción de edición debería registrarse en un stack para deshacer/refacer (Fabric.js permite manejar históricos de estado o implementar propio).
- **Exportar PDF:** Finalmente el periódico debe generar PDF. Opciones: usar **pdfMake** (crea PDF en frontend mediante definición JS) o generar HTML+CSS y convertir con **Headless Chrome** (Puppeteer) en el backend. Por ejemplo, generar HTML de todas las páginas de la edición y hacer `page.pdf()` en Puppeteer. Esto produce un PDF de alta fidelidad (puede usar `@font-face`, etc.).

Comparativa de Canvas:

Librería	Ventajas	Desventajas
Fabric.js	Fácil edición on-canvas (texto enriquecido, grupos). Documentación amplia [38†L11-L18] .	Menos activa que Konva, podría tener limitaciones de rendimiento en proyectos muy grandes.
Konva	Enfoque moderno: capas, Tween, integración React (react-konva) [50†L4-L7] . Muy usado en aplicaciones avanzadas (p.ej. Polotno).	Sintaxis algo más compleja, menos intuitiva para texto enriquecido (requiere librería extra).
Otra (SVG)	Alternativa con D3.js o SVG puro, pero menos performance en dinámicos.	No nativo para canvas; no recomendable para este caso.

6. Flujo editorial y estados de publicación

Definimos estados para cada “Edición” o página de periódico: **Borrador** → **Revisión** → **Aprobado** → **(Publicar/Programar)** → **Publicado**. Este flujo es estándar en CMS de noticias: por ejemplo, Journalify menciona estados “draft, review, approved, published” [27†L14-L16] . El proceso sería:

1. **Borrador:** El periodista crea/edita páginas usando el editor.
2. **Revisión:** Envía a editor jefe. Aquí se le asigna estado “En Revisión”. El editor puede modificar o pedir cambios.
3. **Aprobado:** El editor aprueba la página. En este punto, puede **publicar inmediatamente** o **programar publicación**. (Opcional: generar vista previa compartible).
4. **Programado:** Si se elige, se fija fecha/hora de publicación. El sistema lanzará la página en esa fecha. ApostropheCMS ejemplifica este concepto: “Approved content can be published immediately or scheduled to go live at a specific date and time” [26†L182-L190] .
5. **Publicado:** Finalmente, la página/editado aparece en *latitud18.ultimahora-tv.com*. El front-end público simplemente lee de la base de datos las ediciones publicadas.



- **API de publicación:** Laravel expondrá endpoints para cambiar estado (p.ej. `POST /api/ediciones/{id}/publish`). Al publicar, el backend creará la página en el sitio público o actualizará el sitemap. Se puede usar un servicio en Laravel (Jobs) para agendar publicaciones programadas.

7. Importar/Exportar plantillas

- Se permitirá **crear/guardar/duplicar/importar** plantillas de editor: p.ej. con botones “+ Crear plantilla”, “Guardar como plantilla” (de una edición actual), “Importar plantilla”.
- Proponemos un formato propio `.latitud-template`, que podría ser un archivo JSON (o ZIP con JSON + assets) conteniendo: nombre, diseño JSON, miniatura, metadatos. Ej:

```
{
  "name": "Portada Clásica",
  "design": { ...estructura JSON... }
}
```

- Al importar (`/api/templates/import`), se parsea este JSON y se crea un nuevo registro en `templates` y su versión inicial. Esto facilita compartir diseños entre proyectos.
- Versionado: Cada vez que se edita una plantilla se crea una **nueva versión** en `template_versions`. Se puede visualizar historial similar a ediciones de artículo.
- UI: En el gestor de plantillas (React), listarlas con opciones de duplicar (clonar plantilla existente), editar (que abre el editor con ese diseño) o eliminar.

8. Migración incremental y roadmap

Dado que es una re-arquitectura grande, proponemos un desarrollo por **etapas**:

Entrega	Tareas clave	Est. tiempo	Riesgos
Fase 1.0 (Editor básico)	- Auditoría actual y refactorización inicial. - Modelos/Migraciones de <code>templates</code> , <code>editions</code> (plantillas, páginas). - API básica (CRUD Templates, Ediciones). - Interfaz: componente EditorCanvas con Fabric.js (texto simple, imágenes básicas). - Función “Guardar página” (guarda JSON en BD). - Vista previa PDF básica (pdfmake con contenido mínimo). - Flujos de estado: solo <i>borrador</i> → <i>aprobado</i> → <i>publicado inmediato</i> .	2-3 semanas	Tiempos variables según cantidad de ajustes al código legado; curva de aprendizaje de Fabric.js.
Fase 1.1 (Integración con CMS)	- Integrar editor en la plataforma actual (autorización, layout administrativo). - Asociar noticias: crear acción “Insertar noticia” (título, bajada, foto se traen automáticamente). - Mejorar edición de texto (usar Slate/ProseMirror). - Añadir reglas/guías, panel de propiedades. - Generar PDF con Puppeteer (servidor) o pdfmake (cliente) de la página completa.	1-2 semanas	Manejo de seguridad (subida de imágenes, XSS en HTML generado).

Entrega	Tareas clave	Est. tiempo	Riesgos
Fase 2.0 (Editor avanzado)	- Herramientas avanzadas (capas, grupos, tablas, QR, código personalizado). - Múltiples páginas por edición (ediciones completas). - Flujos editorial completos: revisión/roles, programación. - Import/Export plantillas (.latitude-template). - PDF final de la edición completa (multi-página). - UI de gestión de Plantillas (listar/duplicar/importar).	3-4 semanas	Complejidad de flujo editorial completo; pruebas exhaustivas necesarias para PDF; riesgos en escalado de la UI.
Fase 2.1 (Estabilización)	- Pruebas unitarias/funcionales (Editor, API). - Ajustes de rendimiento (canvas grande, optimizar queries). - Documentación de uso. - Despliegue progresivo (staging → producción).	1-2 semanas	Ajustar según feedback de usuarios; asegurar copias de seguridad de BD antes de migrar.

Roadmap 1.0 vs 2.0: En 1.0 se prioriza editor básico con plantillas fijas, guardado de página y PDF rudimentario. En 2.0 se agrega flujo editorial completo, plantillas importables, edición multi-página y refinamientos (tablas comparativas, leyendas, IA para sugerencias, etc.).

9. Pruebas, seguridad y despliegue

- **CI/CD:** Configurar integración continua con GitHub Actions (o GitLab CI). En cada push: ejecutar `composer test`, `npm run dev` o `build`, y `php artisan migrate --dry-run`. Referencia: CI/CD en Laravel recomienda flujos con test y deploy automatizado [64†L517-L525].
- **Pruebas:** Escribir pruebas unitarias (Modelos) y de integración (controladores de API). Usar **Laravel Dusk** para pruebas de interfaz (login, guardar edición, publicación).
- **Seguridad:** Verificar validaciones de entrada (evitar inyección de JS en plantillas). Usar *sanitización* de textos/HTML. Manejo seguro de permisos: solo usuarios autorizados pueden crear/editar/publish.
- **Backups y escalado:** Asegurar backups periódicos de la DB (por ejemplo, jobs nocturnos que exporten mysqldump). Para subida de imágenes, usar almacenamiento escalable (S3) si se prevé gran volumen. El sistema puede escalar horizontalmente: stateless PHP con Redis para sesiones, balanceador.
- **Despliegue:** Usar métodos de zero-downtime (p.ej. Laravel Envoyer o despliegue Blue/Green). Mantener `.env` con variables sensibles en CI como secretos [64†L417-L426]. Monitorear errores con herramientas (Sentry, Telescope) post-lanzamiento.

10. Archivos y cambios clave en el repositorio

Aunque no tenemos el código original, es necesario indicar qué partes habría que modificar/añadir. A modo de guía:

- **Nuevas rutas:** En `routes/api.php`: rutas para plantillas (`/templates`, `/templates/{id}/versions`) y ediciones (`/editions`, `/editions/{id}/publish`). En `routes/web.php`: ruta para la interfaz del editor (p.ej. `/editor`).
- **Controladores:** Crear `TemplateController` (index, show, store, update, destroy) y `EditionController`. En estos controladores, implementar la lógica de guardado de JSON y cambio de estado. Por ejemplo, en `TemplateController@store` guardar en DB:

```
$template = Template::create([...]);
TemplateVersion::create([
    'template_id' => $template->id,
    'version' => 1,
    'design' => $request->input('design')
]);
```

- **Modelos:** `app/Models/Template.php` y `app/Models/TemplateVersion.php` con relaciones *hasMany*. También `Edition.php` si se maneja Ediciones.
- **Migraciones:** En `database/migrations/****_create_templates_table.php`: agregar columnas según modelo (incl. `json('design')` o mover diseño a versión). En `create_template_versions_table.php`: campos (diseño JSON, versión). Agregar tablas `editions` y `edition_pages` (una edición con múltiples páginas).
- **Frontend React:** Añadir componentes en `resources/js/components/`, p.ej. `EditorCanvas.jsx`, `TemplateManager.jsx`, `EditionList.jsx`. Modificar rutas de React Router o menú en `App.jsx`. Consumir las nuevas APIs.
- **Hooks/Efectos:** En `resources/js/api.js` (o donde estén los clientes axios), añadir funciones para endpoints nuevos (`getTemplates()`, `saveEdition()`, etc.).
- **Vistas Blade/React:** Si Laravel usa Blade para el admin, habrá que insertar `<div id="editor"></div>` y compilar React allí. Si es 100% SPA, se integrará en la SPA.
- **Otros cambios:** Ajustar `User` o `Role` seeders para incluir nuevos permisos; modificar políticas (`app/Policies/*`) para incluir Template; actualizar `UserSeeder` para rol editor.

Cada archivo modificado o agregado sería documentado así:

- `/routes/api.php`: añadir rutas `templates` y `editions`.
- `/app/Http/Controllers/TemplateController.php`: nuevo (gestiona plantillas).
- `/app/Http/Controllers/EditionController.php`: nuevo (publicación de ediciones).
- `/app/Models/Template.php`, `/TemplateVersion.php`, `/Edition.php`: nuevos modelos.
- `/database/migrations/*_create_templates_table.php`: nuevo (estructura de templates).
- `/database/migrations/*_create_template_versions_table.php`: nuevo.
- `resources/js/components/EditorCanvas.jsx`: nuevo (canvas del editor).
- `resources/js/components/TemplateManager.jsx`: nuevo (UI plantillas).
- `resources/js/api/templates.js`: nuevo o actualizado (funciones axios).

En resumen, el repositorio se ampliará con **módulos de plantilla/edición completos**, integrando el editor visual en React, respaldado por Laravel que almacenará diseños en JSON, manejará estados editoriales y publicará contenido automáticamente 【27†L14-L16】 【26†L182-L190】 .

Fuentes: Documentación de Laravel (migraciones JSON 【32†L9-L16】), sitios oficiales de librerías (Fabric.js 【38†L11-L14】 , Konva 【50†L4-L7】 , Slate/ProseMirror 【54†L214-L222】 【56†L22-L27】) y ejemplos de CMS modernos (workflow en Apostrophe 【26†L182-L190】 , Journalify 【27†L14-L16】) han orientado estas recomendaciones. Los diagramas mermaid ilustran la arquitectura propuesta y el flujo editorial (arriba).
